



CSC212 PROJECT PHASE1

Maze Problem



Red team members:

- Fahad Aldakheel 439106720
- Mohammed Alkhunaizan 443102405
- Osama Alajaji 443100980

Division of works:

- Fahad: Wrote the report.
- Mohammed Alkhunaizan: Developed the source code.
- Osama Alajaji: Developed the source code and GUI.



Table of Contents

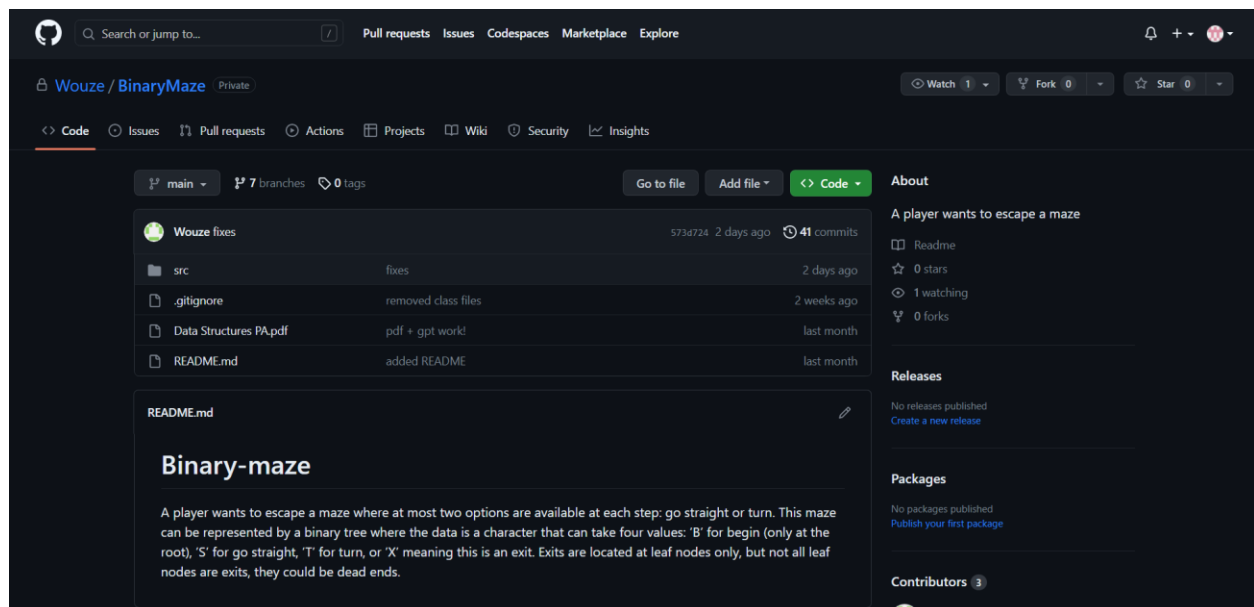
Section 0: Project Journal	2
Section 1: Abstract.....	4
Section 2: Methods description	5
Section 3: Methods Time complexity	7
Section 4: Bonus	9



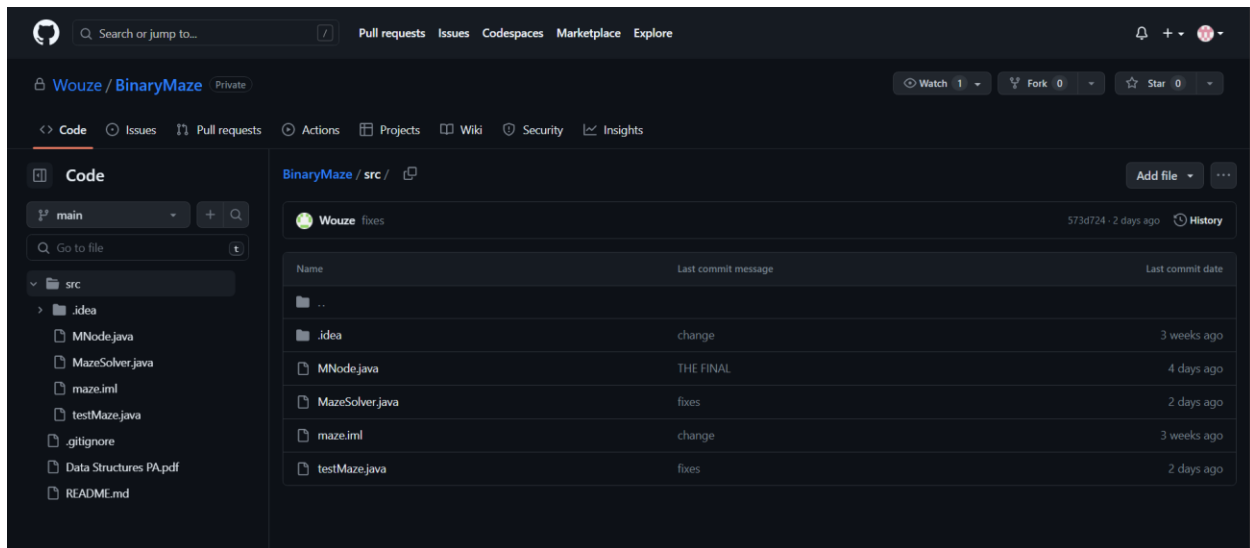
Section 0: Project Journal

On Our first meeting we had an online discussion on CSC212 project. We talked about the requirements and how we can do the project within the timeframe. Furthermore, we talked about our skills and what we can do best, in order to distribute the work between us.

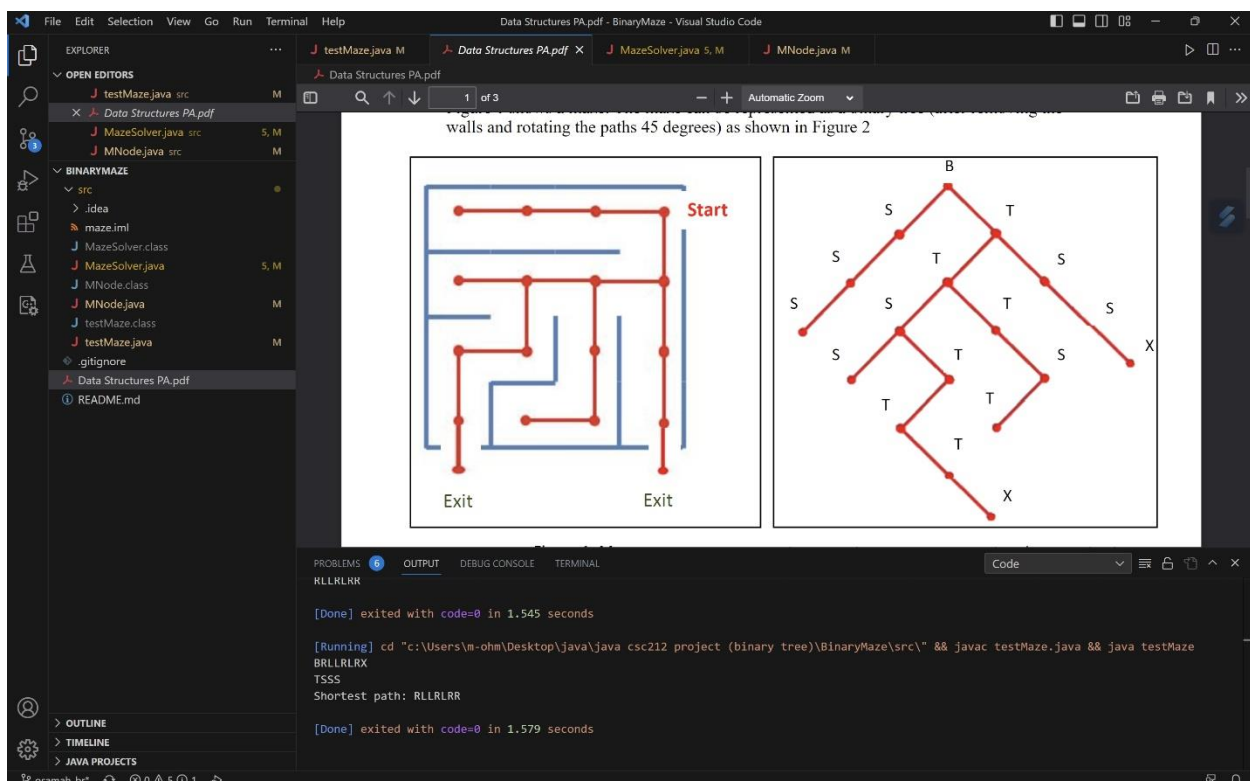
Mohammed have convinced us to start using Github as a tool to upload our work. You can visit our repository here [Github](#)



Using Github was challenging at first, but we got used to it with time.



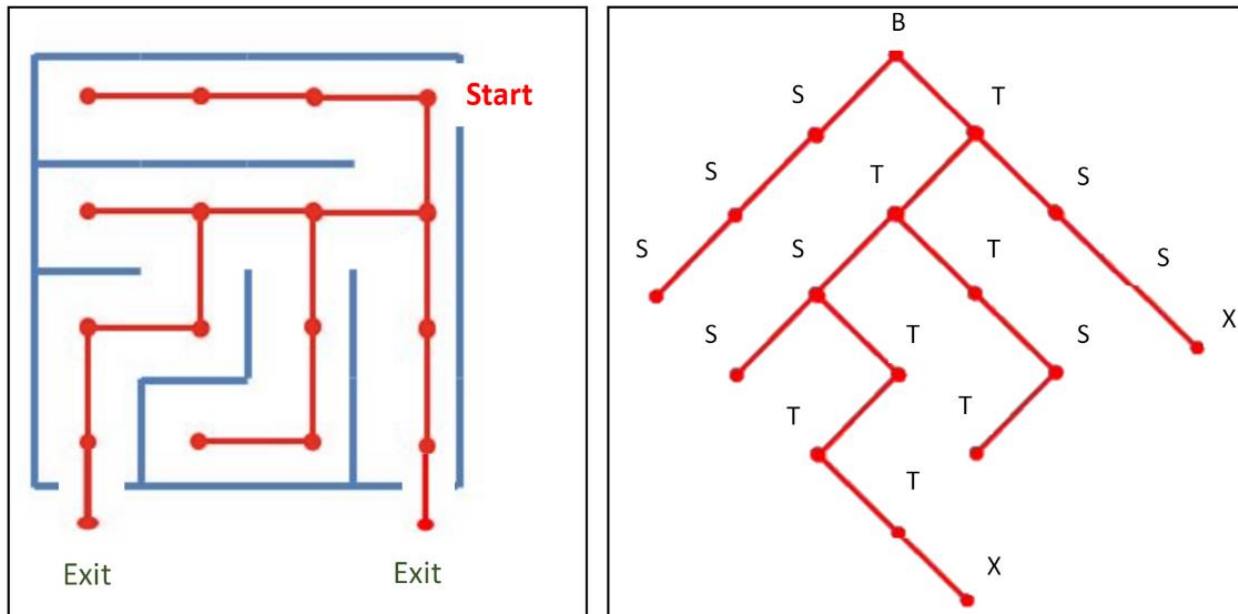
Three weeks in, the developer team had a breakthrough in the project.



They completed the source code on Tuesday 5/16.

Section 1: Abstract

The programming project will provide students with practical experience working with data structures. Implementation and utilize a binary tree data structure. Three methods are required to be implemented.



Section 2: Methods description

❖ Method **Follow**

- The **follow** method tests if a given path starting from a specified node in a maze is valid. It iterates through each character in the path string, checking if the corresponding directions and data values in the maze are valid. If any invalid condition is encountered, it returns false. If the entire path is successfully traversed without encountering invalid conditions, it returns true.

❖ Method **escape**



The **escape** method searches for an exit in a maze using preorder traversal. It checks if the current node is an exit and returns true if it is. Otherwise, it recursively calls itself on the left and right child nodes. If an exit is found in either subtree, it returns true. If no exit is found in the entire maze, it returns false.

❖ Method **shortest**

- The **shortest** method returns the shortest exit path from the root using recursion and depth-first search. The path is a string of letters for directions. For example, "T-S-S-X"

❖ Method **findShortest**

- The **findShortest** method returns the shortest exit path from a node using recursion. It adds “L” or “R” to the path for left or right child. It returns the shorter path or the non-empty one.

❖ Method **addPathToTree**



The **addPathToTree** method creates the maze nodes within the tree based on a given path for example “SSSTTTSSSTTX”. It translates the path into a left-right format and iterates over each character. It creates left or right nodes, and marks nodes as exits when encountering an 'X' character.

❖ Method **TranslateToLeftRight**



The **translateToLeftRight** method modifies a path string by replacing 'S' with 'L' or 'R' based on the current side value, and 'T' with 'R' or 'L' while updating the side value. Any other characters are left unchanged. The modified path string is returned.

❖ Method **translateToStrightTurn**

- The **translateToStrightTurn** method takes a string **RL** as input and converts it to a modified version called **path** by replacing consecutive identical characters with 'S' and alternating characters with 'T'. The modified **path** string is then returned.

Section 3: Methods Time complexity

- Method **Follow**: This method iterates over characters in the path string and checks if the current MNode has a child node for the next character in the path. The number of operations performed is related to the length of the path string, the time complexity will be **$O(n)$** .
- Method **Escape**: The method performs a pre-order traversal of the tree entering each node once. The method returns true if it finds an exit, or false if no exit were found, so it does not visit all the nodes in worst case. The worst case is when the exit does not exist, or it is the last node in tree to be visited. The time complexity is **$O(m)$** . m = number of nodes in the tree.
- Method **Shortest**: The method calls the **findShortest** which has a time complexity of $O(m^2)$. The other operations are constant, such as checking if the root is null or returning an empty string. The time complexity is **$O(m^2)$** . m = number of nodes in the tree



- Method **findShortest**: The method needs to recursively call itself on the left and right sides of a node t . The method is called when we go down in the tree. In order to visit every node in the tree. the worst case, where the tree is not balanced, every node will call itself. So, the method time complexity will be **$O(m^2)$** time, m = number of nodes in the tree
- Method **TranslateToLeftRight**: The method iterates over the characters in path and performs constant-time operations for each character. The time complexity is **$O(n)$** , n = length of the path string.
- Method **translateToStrightTurn**: The method iterates over the characters in RL and performs constant-time operations for each character. The time complexity is **$O(n)$** , n = length of RL string
- Method **addPathToTree**: The method traverses the given path and creates nodes accordingly. For each character in the path, it either creates a new node or assigns a value to an existing node. The number of nodes created depends on the structure of the given path. The time complexity to the number of nodes created in the tree, which is denoted as **$O(n)$** . n = number of paths



Section 4: Bonus

Osama Alajaji has been working on a graphical user interface “GUI”, and he also made a game using the previous methods with implementing the maze concept. Here’s some screenshots for the game.

